

This Ship Is on Fire

Project 1: Strategies, Experiments, and Failure Analysis

Derek Sun (dds168), Prashasth Turaga (pnt31)

October 9, 2026

1 The Problem and the Four Bots

The goal is to get the bot to the fire-suppression button on a randomly generated ship before it burns. The ship is a square grid, and the bot can move to an adjacent open cell or stay put. Fire spreads every turn: a cell catches fire with probability $1 - (1 - q)^K$, where K is the number of burning neighbors. The TA told us the button itself can't catch fire, but its neighbors can burn and cut the bot off from it.

Each turn the bot picks a move and makes it. If it walks into fire, it fails. If it reaches the button, it wins right away, so the last step doesn't need to survive another fire update. Otherwise the fire spreads, and the bot fails if its cell ignites.

- **Bot 1** plans one shortest path at the start (avoiding the initial fire) and just follows it.
- **Bot 2** replans the shortest path every turn, avoiding the current fire.
- **Bot 3** replans every turn and tries to avoid both the fire and every cell next to it (not counting the button). If no such path exists, it falls back to Bot 2's rule.
- **Bot 4** is our own design, explained next.

2 Bot 4 Design

All of the shortest-path bots share a problem: a cell that's safe right now can be on fire by the time the bot gets there. So with Bot 4 we wanted to pick the path with the lowest predicted fire risk at the moment we'd actually be standing in each cell. Unlike Bot 3's wider buffer, this uses q , how fire moves through the hallways, and when we'd arrive at each cell.

Forecasting the fire: Let $P_t(c)$ be our estimate of the probability that cell c is burning after t more fire updates. Burning cells start at 1 and everything else starts at 0. For a cell that can ignite, we update

$$P_{t+1}(c) = P_t(c) + (1 - P_t(c)) \left[1 - \prod_{n \in N(c)} (1 - q P_t(n)) \right]$$

Walls and the button can't ignite. The product estimates the chance that none of the neighbors light the cell, and we treat the neighbors as independent. That isn't exactly true, since nearby fires are correlated, but it's a cheap way to forecast without simulating tons of fires every turn. We only use it to rank routes, not as an exact survival probability.

Turning risk into a cost: If we’d be in cell c after move t , the cost of that cell is

$$w_t(c) = -\log(1 - P_t(c)).$$

Maximizing a product of survival probabilities is the same as minimizing a sum of negative logs, so this lets us add up risk along a path. Safe cells cost close to nothing, and cells that are almost certainly burning cost a huge amount (walls and cells with $P_t \geq 1 - 10^{-12}$ get infinite cost). The button costs zero because we win as soon as we reach it. There’s no separate weight for distance, so the bot will take a longer route whenever its total risk is lower, even by a tiny amount. When two arrivals tie, we take the earlier one.

Searching through time: First we compute the current shortest path that avoids fire, with length L moves. Our planning horizon is $H = L + 20$, so we have some room to detour without the search blowing up. The 20 is just a number we picked, not something we tuned. Let $C_t(c)$ be the lowest total risk cost for getting to c after t moves. The bot’s own cell starts at 0 and everything else at infinity, and each layer is

$$C_t(c) = w_t(c) + \min_{n \in N(c)} C_{t-1}(n).$$

Bot 4 picks the lowest-cost button arrival over moves 1 through H , backtracks to find the first move, and makes only that move. Next turn we redo the whole forecast from the actual fire, so we never get stuck following a prediction that turned out wrong. The search lets us revisit cells but doesn’t explicitly consider staying.

Two special cases: if the fire has already cut the bot off from the button, Bot 4 stays, since fire never goes away. If a safe path exists right now but every forecast route has infinite cost, the forecast is probably just too pessimistic, so we take the first step of the plain shortest path.

Efficiency: We use numpy shifts to compute the four-neighbor forecast and the best-predecessor step over the whole grid at once, and we cache the Boolean grid between turns. Each decision takes $O(HD^2)$ time and $O(HD^2)$ memory for the stored layers, so the horizon controls both how the bot behaves and how much computation it uses. In a current benchmark of 15 complete trials per bot, Bots 1–4 averaged 0.102, 0.114, 0.117, and 0.219 seconds per trial on 60×60 ships. Bot 4 averaged 0.900 seconds on 80×80 ships, so that size was manageable but much slower. We used 60×60 ships for the experiments to collect enough trials in a reasonable amount of time.

3 Experiments and Results

Setup: For the full sweep we generated 100 ships of size 60×60 for each $q \in \{0.0, 0.1, \dots, 1.0\}$. In each scenario we randomly placed the bot, button, and initial fire on three different open cells, then ran all four bots, for 4,400 runs total. All four bots got the same ship, starting positions, and fire seed, so they all saw the same fire. Since bot movement doesn’t change how the fire spreads, this makes the comparison fair. We reused the same 100 scenarios across q values.

We also ran a focused follow-up at $q = 0.3$ with 200 new scenarios, which together with the 100 from the sweep gives 300 per bot. In total that’s 5,200 distinct runs. The raw data stores the seeds, positions, and a 5,000-turn cutoff, which no run reached. The whole collection took about 17 minutes.

Full-range results:

q	Bot 1	Bot 2	Bot 3	Bot 4
0.0	100%	100%	100%	100%
0.1	96%	99%	99%	99%
0.2	91%	96%	97%	99%
0.3	89%	94%	95%	96%
0.4	82%	86%	86%	88%
0.5	79%	81%	81%	82%
0.6	76%	77%	77%	78%
0.7	69%	69%	69%	71%
0.8	66%	66%	66%	68%
0.9	62%	63%	63%	64%
1.0	56%	56%	56%	57%

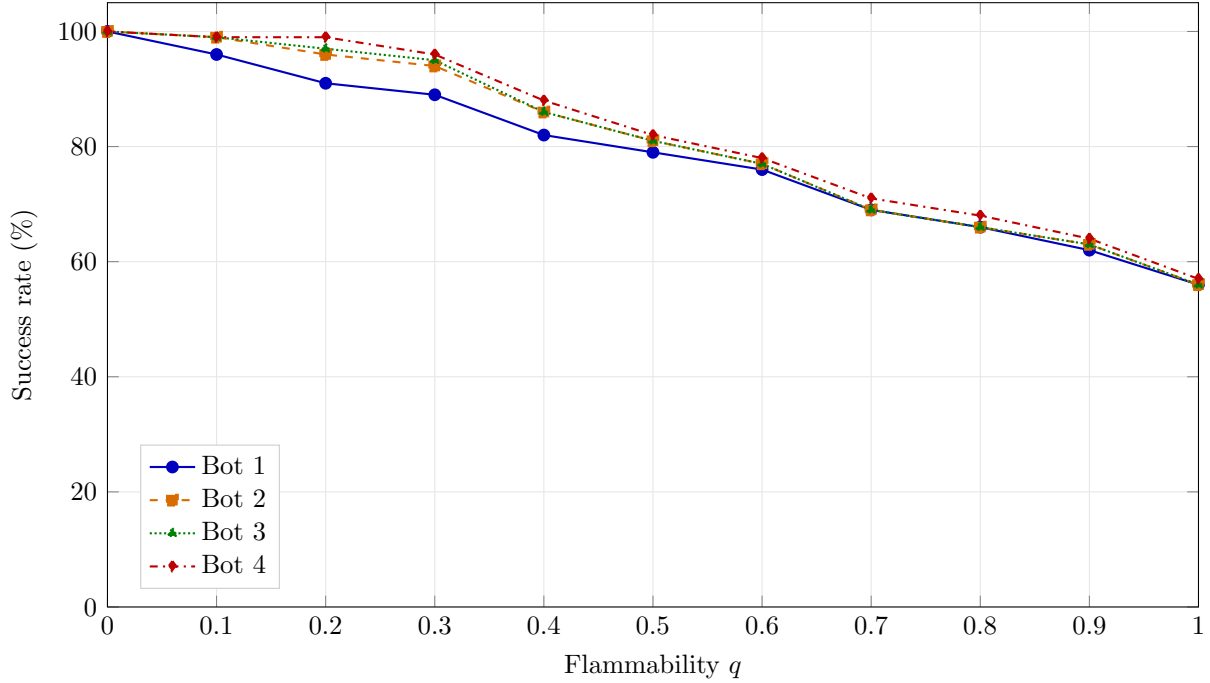


Figure 1: Success rate versus flammability on 60×60 ships with a fireproof button. Each point uses 100 scenarios, and all bots share the same initial conditions and fire. The separate $q = 0.3$ follow-up uses 300 scenarios and is reported in the focused table.

At $q = 0$ every bot wins. At low q , most routes finish before the fire matters. At $q = 0.2$ – 0.4 , Bot 4 is on top in Figure 1. At high q the curves squeeze together, which makes sense: the fire spreads so fast that there’s no time for clever detours, and it mostly comes down to whether the bot started closer to the button than the fire did. At $q = 1$, Bots 1–3 win 56 times and Bot 4 wins 57, which is just one scenario.

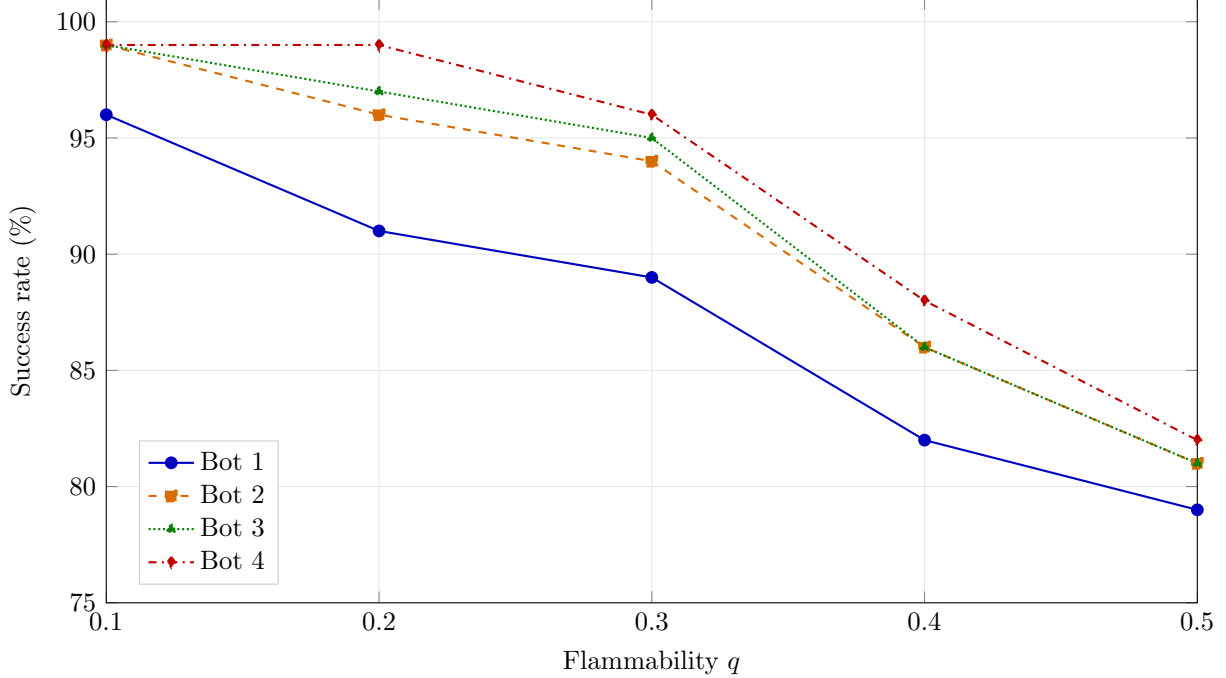


Figure 2: Zoom of Figure 1 over $q = 0.1$ – 0.5 , using the same 100 scenarios per point. The vertical axis is cut off at 75–101% to show the small differences.

Focused comparison at $q = 0.3$.

Bot	Successes at $q = 0.3$	Success rate
Bot 1	264/300	88.00%
Bot 2	275/300	91.67%
Bot 3	278/300	92.67%
Bot 4	282/300	94.00%

This is where Bot 4 outperforms the other three bots. It had the highest raw success rate, and since every bot ran on the same 300 ships with the same fire, we can also compare them ship by ship. Bot 4 survived 4 ships that Bot 3 lost and never lost a ship that Bot 3 survived. Against Bot 2 it was 8 ships to 1, and against Bot 1 it was 19 to 1. The gap over Bot 3 is still small, and 300 trials isn’t enough for us to say Bot 4 is definitely better than Bots 2 and 3. We chose $q = 0.3$ before running these experiments, and we didn’t adjust Bot 4 based on the results.

4 Why the Bots fail

All of these cases use $q = 0.3$ and the fireproof-button rules. Coordinates are zero-based and written as (row, column). We replayed each trial and checked that the saved outcomes and moves were legal. To see whether a failure was avoidable, we also searched every reachable state (including staying) using the fire that actually happened. This gives the optimal path for that specific fire, which no bot could know in advance.

Trial	Bot 1	Bot 2	Bot 3	Bot 4
50	Burns, 38	Burns, 61	Burns, 60	Success, 79
249	Success, 68	Success, 68	Burns, 100	Burns, 81
0	Burns, 49	Burns, 82	Burns, 82	Burns, 82

Trial 50: the risk-aware detour ends up working out for Bot 4. On turn 18, at (43,19), Bots 3 and 4 make different moves even though they see the same fire. Bot 3 goes to (43,20) and Bot 4 goes to (42,19). Bot 4 reaches the button in 79 turns, and the optimal path for that fire is also 79 moves. Bot 3 burns on turn 60.

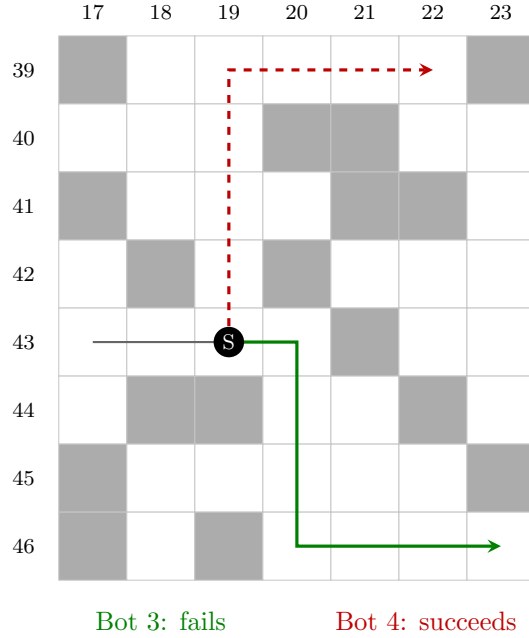


Figure 3: Local paths in Trial 50, $q = 0.3$. Rows 39–46 and columns 17–23 are cropped from the saved grid, and gray squares are walls. The gray segment shows the shared moves 15–17. At S, (43,19), both bots see identical fire. Green solid and red dashed paths show Bot 3’s and Bot 4’s moves 18–24. The burning cells and button are outside this portion of the grid. Bot 3 burns on turn 60 and Bot 4 reaches the button on turn 79.

Bot 1 walks into fire at (50,31) on turn 38, even though a 34-move path around the current fire still existed. That one was avoidable. Bots 2 and 3 get cut off from the button by turn 51, so they stay put and eventually burn. The button never burns; the fire just surrounds it.

Trial 249: too much detouring causes Bot 4 to lose a game it could’ve won. Bots 1 and 2 both succeed in 68 turns, so a winning strategy clearly existed. Bot 4 first splits from Bot 1 on turn 23 at (20,15), going to (20,14) instead of (21,15). Bot 3 follows the direct route longer and splits on turn 55. Bot 4 burns on turn 81 after stepping into (44,34), which catches fire that turn. Right before that move, the button was only two moves away. Bot 3 gets cut off and burns on turn 100. The optimal path for this fire is 68 moves.

So Bot 4 spent too long chasing a lower-risk route. To test this, we reran Trials 50 and 249 changing only the horizon slack:

Extra horizon moves	Trial 50	Trial 249
0	Burns, 61	Success, 68
5	Burns, 61	Success, 68
20 (default)	Success, 79	Burns, 81
40	Success, 79	Burns, 81

A longer horizon helps in one trial and hurts in the other. We kept the default at 20, since we didn’t want to tune it on two hand-picked seeds.

Trial 0: no way out. Searching every reachable state with the actual fire finds no escape at all (it runs out of states on turn 266). Even knowing the fire in advance couldn’t save the bot here. Bots 2–4 lose their route to the button by turn 39 and burn later, and Bot 1 keeps following its plan into fire.

5 What surprised us

We expected Bot 4 to help most when the fire spreads fast enough to matter but slowly enough that detours are still useful. The graph matches that, since nearly every bot wins at low q and the curves bunch up at high q .

At first we were worried the bots were basically the same. An early version of our demo had all four bots burn on the same first turn, and a small 10×10 test gave overlapping curves. Controlled layouts and full-path replays showed they do make different decisions: Bot 3 takes buffer-driven detours, and Bot 4 sometimes picks a different move from Bots 2 and 3 with identical fire. In one controlled 6×6 case, Bots 2 and 3 moved from (2,3) to (3,3), while Bot 4 moved to (2,2) after seeing the same fire. If we were to just look at success rates, we wouldn’t have seen those differences, especially when different choices end in the same result.

The TA’s clarification that the button can’t burn also changed our process. Our first experiments let the button burn, so we separated those out and recollected everything. None of the old numbers are mixed in with the results here.

The biggest surprise was how small Bot 4’s edge over Bot 3 is. A more expensive strategy isn’t automatically much better. Trial 249 shows the downside, since a low-risk detour can lose a game that the direct route wins, and Trial 50 shows the upside. This shows us that a good bot has to balance risk against urgency, instead of assuming that more lookahead is always better.

6 Ideal Bot and Computation Time

An ideal bot would use everything it can see: the grid, its position, the button, the burning cells, and q . It would consider every action, including staying, and plan over all the ways the fire could play out, adapting its later moves to what it sees. That’s different from committing to a single forecast route. The problem is that exact planning is huge, because any subset of open cells could be burning.

A more practical version would simulate a bunch of random fire futures for each candidate move and score each one with a fast escape policy. That would capture the fire correlations that our forecast ignores, though it costs more time and adds sampling noise. We didn’t implement it.

In our simulator the fire waits for the bot to finish deciding, so slow computation never directly causes extra spread, and our timings don’t show a real-time penalty. In a real system the bot would need a deadline: compute a quick fallback move first, then improve it if there’s time. When fire is

right next to the bot, a fast move matters more than a perfect plan that arrives late. When the fire is far away, extra computation is worth it.

The hint about guaranteed wins also suggests a way to collect data faster. Fire can only spread one edge per update, so if a route reaches every cell strictly before the earliest possible ignition and then reaches the button, it's safe even under the fastest possible spread. This certifies a specific route without running randomized trials, but it doesn't certify every route a bot might pick.